HW#4 RTOS Analysis

Chun-Jen Tsai
NYCU
11/21/2025

Homework Goal

- In this homework, you will analyze the performance of a real-time OS (RTOS), FreeRTOS, for multithreading
- Your tasks:
  - Trace the OS kernel code and analyze how thread management and synchronization are done
  - Measure the context-switching overhead of the application (by adding hardware counters in Aquila)
- You should upload your report to E3 by 12/8, 23:55.
  - PDF format only, 3 pages max. No demo this time.

FreeRTOS

- FreeRTOS is a C-based real-time OS kernel for embedded systems
  - Developed by Richard Barry in 2003
  - Barry joined Amazon Web Services (AWS) and passed the stewardship of the project to AWS in 2017
  - The project adopts MIT License
- We used FreeRTOS v10.4.6 in this HW:
  - All RISC-V unrelated sources are removed (way too big)
  - Official site https://github.com/FreeRTOS/
  - No need to modify source code of FreeRTOS for Aquila

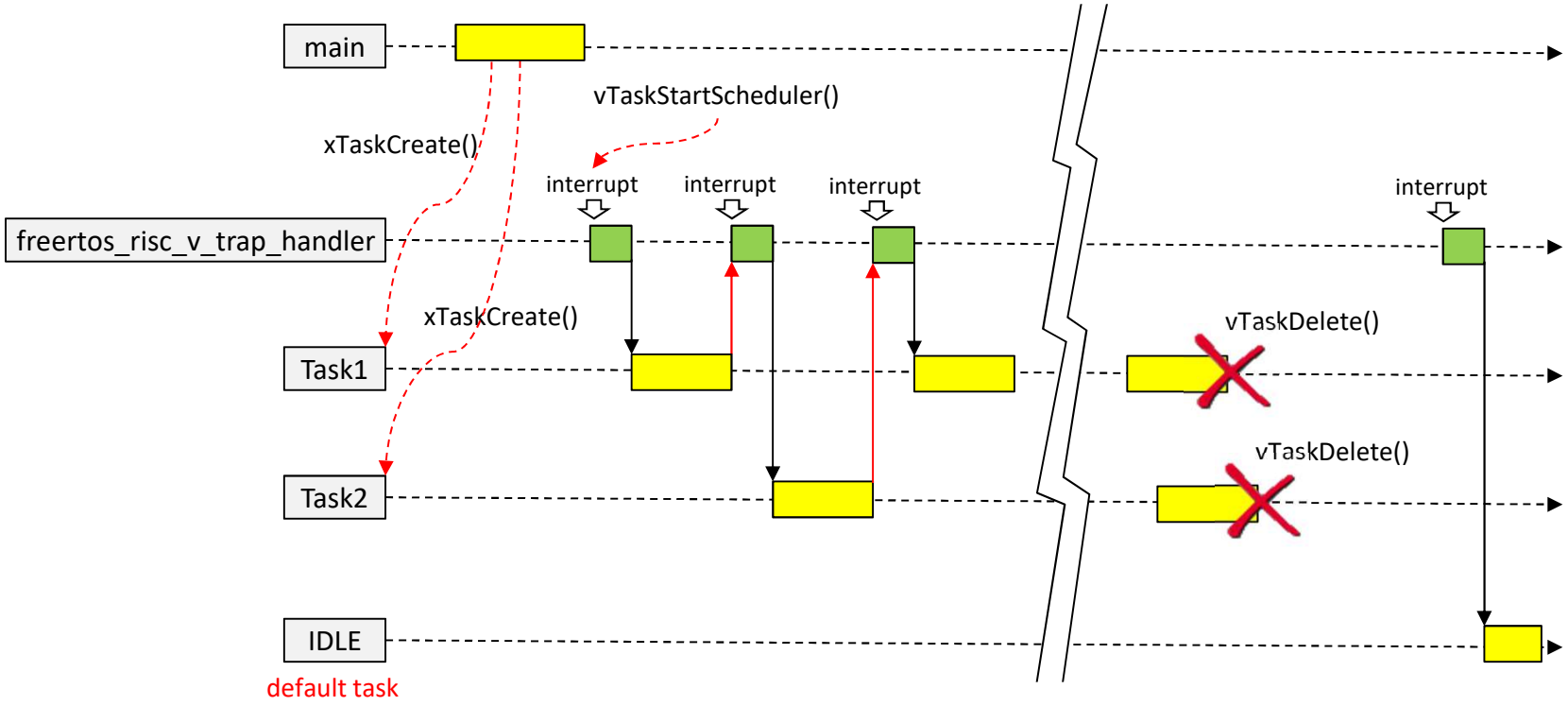
FreeRTOS Multithreading API

- C has no standard multithreading API until ISO/C11
  - The most popular API is still the POSIX pthread API
  - FreeRTOS has its own multithread API, but quite easy to learn
  - There is a third-party patch for FreeRTOS to support pthread
- FreeRTOS multithread example:

```
/* Two-thread creation for FreeRTOS */

int main(void)
{
    int prm1 = 1, prm2 = 2;
    xTaskCreate(Task_Handler, "Task1", 256, (void *) &prm1, 3, NULL);
    xTaskCreate(Task_Handler, "Task2", 256, (void *) &prm2, 4, NULL);
    vTaskStartScheduler();
}

void Task_Handler(void *pvParam)
{
    for (int idx = 0; idx < 10/(int) *pvParam; idx++) {
        printf("\nThis is Task%d.\n", (int) *pvParam);
        vTaskDelay(1000/portTICK_PERIOD_MS);  // sleep a while.
    }

    vTaskDelete(NULL); /* Thread ends, delete it from the task queue. */
}
```

Multithread Behavior of FreeRTOS

- If an application creates two threads, Task1 and Task2, the life-cycle of the system is as follows:



The Test Program for HW#4

- For this homework, the test program, `rtos_mmul.c`, performs parallel matrix multiplication
  - The program was written by DeepSeek
- You can configure the data size and # thread:

```
#define N 128             // Size of matrices.
#define NUM_TASKS 4       // Number of tasks (threads).
#define STACK_DEPTH 1024 // Stack size of each task.

// Shared task parameter structure.
typedef struct {
    float* A;
    float* B;
    float* C;
    int n;
    int start_row;
    int end_row;
    SemaphoreHandle_t completion_sem;
} matrix_task_params_t;
```

Source Tree Structure of HW#4

```
rtos_mmul/
├── FreeRTOS/
│   ├── Demo/
│   ├── License/
│   ├── README.md
│   ├── Source/
│   │   ├── croutine.c
│   │   ├── event_groups.c
│   │   ├── include/
│   │   ├── list.c
│   │   ├── portable/
│   │   │   ├── Common/
│   │   │   ├── GCC/
│   │   │   │   └── RISC-V/
│   │   │   │       ├── Documentation.url
│   │   │   │       ├── chip_specific_extensions/
│   │   │   │       ├── port.c
│   │   │   │       ├── portASM.S
│   │   │   │       ├── portmacro.h
│   │   │   │       └── readme.txt
│   │   │   ├── MemMang/
│   │   │   └── readme.txt
│   │   ├── queue.c
│   │   ├── stream_buffer.c
│   │   ├── tasks.c
│   │   └── timers.c
│   └── Test/
├── FreeRTOSConfig.h
└── Makefile, rtos_mmul.c, rtos_mmul.ld
```

277 directories, 1446 files

portASM.S → Timer interrupt handling routine.

tasks.c → Multi-thread managers.

FreeRTOSConfig.h → FreeRTOS configurations for application.

After "make", a `build/` directory would contain the `rtos_mmul.map` and `rtos_mmul.objdump` files.

The executable `rtos_mmul.elf` will be in the top directory

Computation Task of HW#2

```
void matrix_multiply_task(void* pvParameters) {
    matrix_task_params_t* p = (matrix_task_params_t*)pvParameters;
    int n = p->n;

    taskENTER_CRITICAL();
    printf("Column %d to %d task begins...\n", p->start_row, p->end_row - 1);
    taskEXIT_CRITICAL();

    for (int i = p->start_row; i < p->end_row; i++) {
        for (int j = 0; j < n; j++) {
            float sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += p->A[i * n + k] * p->B[k * n + j];
            }
            p->C[i * n + j] = sum;
        }

        // if (i % 10 == 0) { taskYIELD(); }
   }

    taskENTER_CRITICAL();
    printf("Column %d to %d task finished.\n", p->start_row, p->end_row - 1);
    taskEXIT_CRITICAL();

    // Post the completion of a task.
    xSemaphoreGive(p->completion_sem);
    vTaskDelete(NULL);
}
```

FreeRTOS Synchronization API

- There are 2 pairs of synchronization API in FreeRTOS:
  1. `xSemaphoreTake() / xSemaphoreGive()`
  2. `taskENTER_CRITICAL() / taskEXIT_CRITICAL()`

- Note that there is no mutex API since a mutex is treated as a binary semaphore in FreeRTOS

- We use the 1st pair to signal the end of a computation task, and the 2nd pair to protect UART (i.e. `printf()`)
  - What are the algorithmic differences of these two pairs of APIs?

Default Execution

- The output of the program is as follows:

```
==========================================================================
Copyright (c) 2019-2025, EISL@NYCU, Hsinchu, Taiwan.
The Aquila SoC is ready.
Waiting for an ELF file to be sent from the UART ...

Program entry point at 0x800015A4, size = 0xFA94.
--------------------------------------------------------------------------
FreeRTOS Parallel Matrix Multiplication
Matrix size: 128 x 128
Number of tasks: 4
Matrices initialization completed.
Column 0 to 31 multiplication task begins...
Column 32 to 63 multiplication task begins...
Column 64 to 95 multiplication task begins...
Column 96 to 127 multiplication task begins...
Column 96 to 127 multiplication task finished.
Column 32 to 63 multiplication task finished.
Column 64 to 95 multiplication task finished.
Column 0 to 31 multiplication task finished.
All tasks are completed!
Computation done! Time spent: 1222 ticks.
Program finished.
```

One tick equals one time slice duration.

Context-Switch Overhead (1/2)

- A preemptive multi-tasking OS uses timer interrupts to transfer CPU usage from one thread to another
  - The context-switch overhead is inversely proportional to the time quantum (time slice)
  - The default value of FreeRTOS time quantum is 10 msec
- Context-switching impacts the efficiency of data cache
  - Context switching may cause serious cache misses if the total working set size of all tasks is larger than the data cache size
  - Context-switch overhead must include cache misses overhead

Context-Switch Overhead (2/2)

- The context-switch time can be measured by counting the cycles between the arrival of a timer interrupt and the execution of the new task
- You can try different time quantum to see the % of cycles of context switching vs. total execution cycles
- To analyze the context-switch overhead related to data cache, you can analyze the miss rate of multiple threads vs. single thread
- The total execution time of multiple threads vs. *no multi-threading* gives you the total context switch overhead

The Timer Interrupt Device

- In Aquila, the module `clint` is used to provide timer interrupts and software interrupts
- `Clint` has three registers (these reg
  - `mtime`: 64-bit counter of timer ticks
  - `mtimecmp`: Upper-threshold to trigger a timer interrupt
  - `msip`: a 32-bit register to trigger a software interrupt
- FreeRTOS will update `mtimecmp` to setup the next context-switch time (based on time quantum duration)

Changing Time Quantum

- In FreeRTOS, time quantum is configured by the header file: `FreeRTOSConfig.h`
  - The default time quantum is 10 msec:

```
...

#define CLINT_CTRL_ADDR                    ( 0xF0000000UL )
#define configMTIME_BASE_ADDRESS           ( CLINT_CTRL_ADDR + 0x0UL )
#define configMTIMECMP_BASE_ADDRESS        ( CLINT_CTRL_ADDR + 0x8UL )

#define configUSE_PREEMPTION               1
#define configUSE_IDLE_HOOK                0
#define configUSE_TICK_HOOK                1
#define configCPU_CLOCK_HZ                 ( ( uint32_t ) ( 50000000 ) )
#define configTICK_RATE_HZ                 ( ( TickType_t ) 100 )
#define configMAX_PRIORITIES               ( 7 )
#define configMINIMAL_STACK_SIZE           ( ( uint32_t ) 100 )
#define configTOTAL_HEAP_SIZE              ( ( size_t ) ( 12 * 1024 ) )

...
```

Comments on the Homework

- The HW is about multi-threading analysis of RTOS
- Grade is totally based on your analysis of the RTOS:
  - The context switching behavior analysis:
    - Description of the switching algorithm
    - Context switching overhead vs. time quantum size
    - *Impact to data cache due to context switching (You may chage data size to illustrate the impact)*
  - The synchronization behavior analysis
    - The algorithmic description of `xSemaphoreTake/Give` and `taskENTER/Leave_CRITICAL`
    - The overhead of a synchronization operation